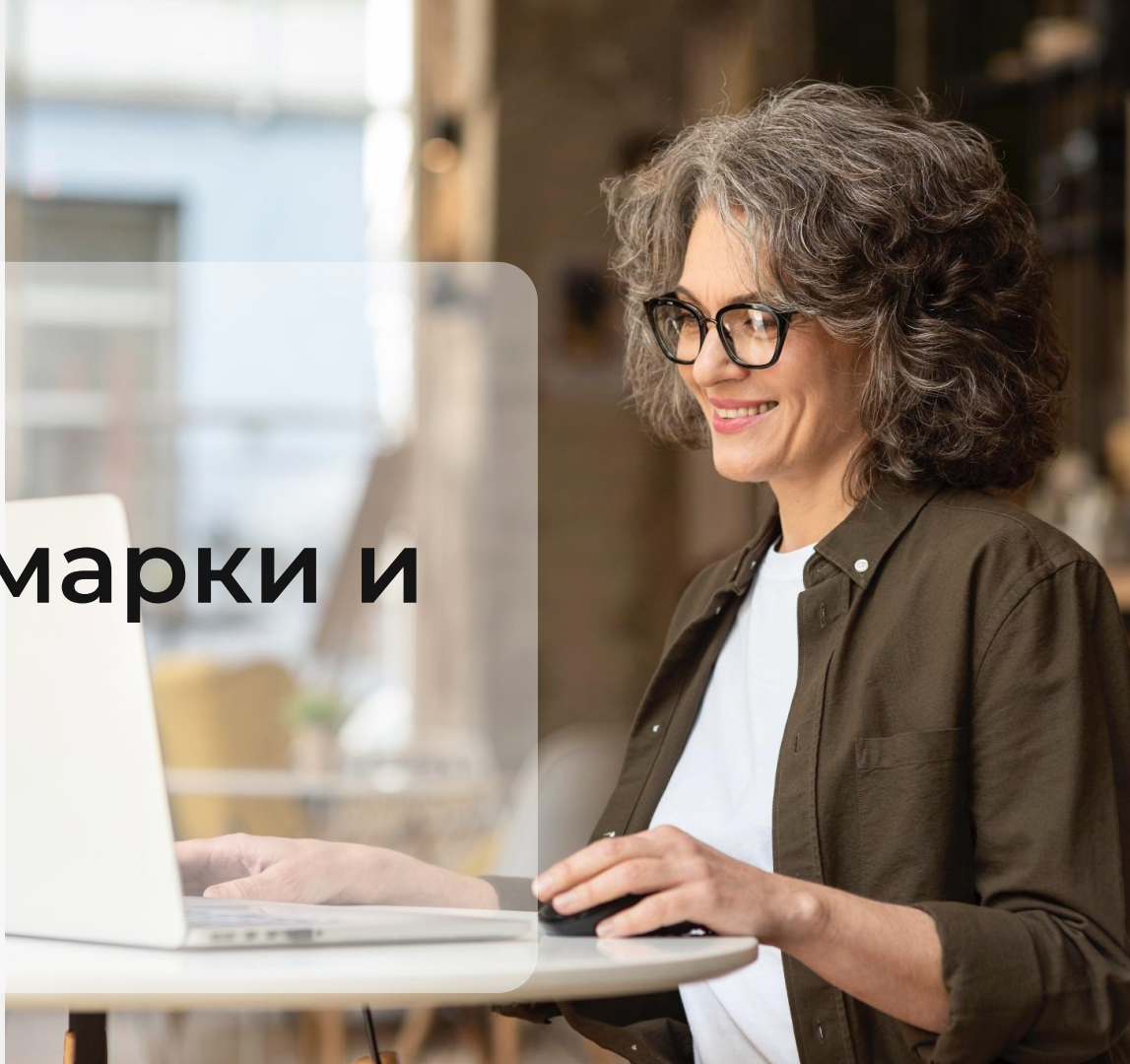


Python
for DA

Polars: бенчмарки и LazyFrame



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Пример
-  Join
-  Asof Join
-  Merge_sorted
-  Конкатенация
-  Melt

План занятия

- Ленивые вычисления
- Создание LazyFrame
- Преобразования LazyFrame
- Визуализация планов выполнения в Polars
- Параметр streaming
- Сравнение с pandas. Бенчмаркинг
- Работа с несколькими файлами



ОСНОВНОЙ БЛОК





Ленивые вычисления





Ленивые вычисления

Это вычислительная стратегия, которая откладывает вычисление выражений до тех пор, пока не понадобятся их значения.

Преимущества ленивых вычислений



Оптимизированная производительность



Эффективность использования памяти



Гибкость в обработке данных



Улучшенная обработка ошибок

Ленивые вычисления



Polars использует ленивые вычисления для анализа и оптимизации всего конвейера, гарантируя, что данные будут обработаны наиболее эффективным способом



LazyFrame

Это структура данных в Polars, которая представляет собой отложенные вычисления или серию преобразований, применяемых к набору данных.



ВОПРОСЫ





Создание LazyFrame

От DataFrames к LazyFrames

```
import polars as pl

df = pl.DataFrame({
    "name": ["Alice", "Bob", "Charlie"],
    "age": [25, 30, 35],
    "city": ["New York", "Los Angeles", "Chicago"]
})

lazy_df = df.lazy()
```

Чтение данных в LazyFrames

```
lazy_df_from_csv = pl.read_csv("events.csv").lazy()
```

Создание LazyFrames с нуля

```
lazy_df_from_scratch = pl.DataFrame({  
    "id": [1, 2, 3],  
    "value": [10, 20, 30]  
}).lazy()
```

Операции scan_

```
lazy_df_from_csv_scan = pl.scan_csv("very_large_dataset.csv")
```

```
lazy_df_from_parquet_scan = pl.scan_parquet("very_large_dataset.parquet")
```

Операции scan_

```
result_df = lazy_df_from_csv_scan.collect()
```



ВОПРОСЫ





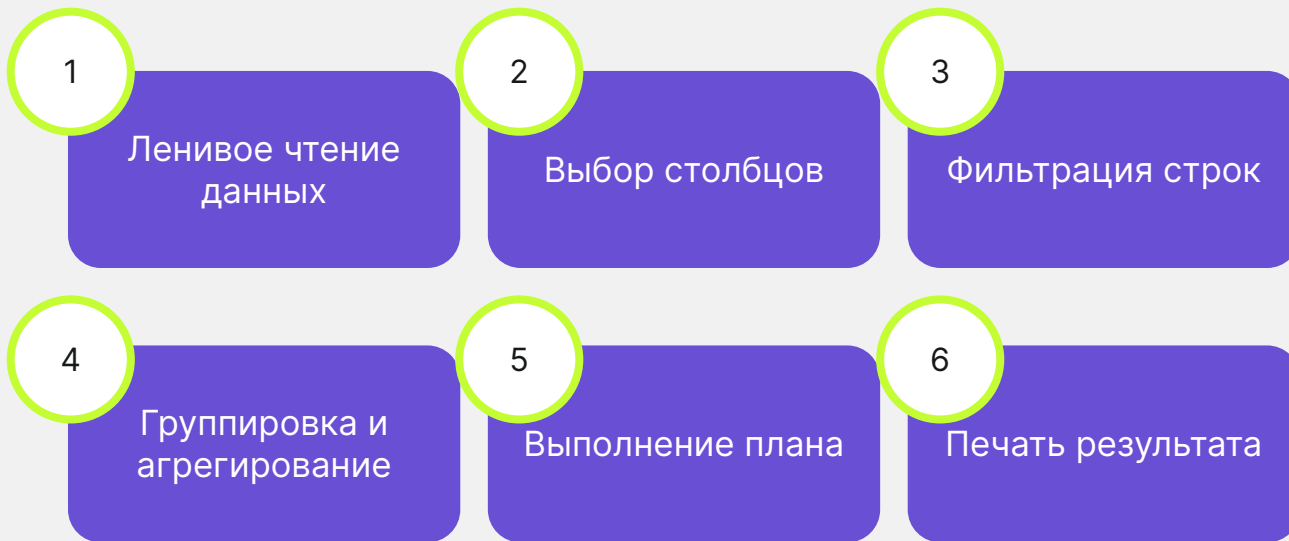
Преобразования LazyFrame

Датасет

```
result_df = (
    pl.scan_csv("events.csv")
    .select(['brand', 'event_type', 'price'])
    .filter(pl.col('event_type') == 'view')
    .group_by("brand").agg([pl.col("price").mean().alias("average_price")])
).collect()

print(result_df)
```

Разбор кода





ВОПРОСЫ





Визуализация планов выполнения в Polars



План выполнения

Это представление последовательности операций, которые планируется выполнить над набором данных. При использовании ленивых вычислений эти операции не выполняются сразу, а собираются в вычислительный граф.



Вычислительный граф

Описывает зависимости между операциями и порядок их выполнения. Визуализация графа может дать ценные сведения об эффективности и структуре конвейера обработки данных.

Зачем визуализировать планы выполнения

Возможности оптимизации

Отладка

Обучение и понимание

Визуализация планов выполнения



Для использования методов визуализации следует установить [Graphviz](#) и добавить путь к бинарным файлам (\bin) в системную переменную PATH.



`.show_graph()`

Это метод, который можно вызвать на LazyFrame для создания визуального представления плана выполнения. Этот метод выводит график, который показывает последовательность операций и их взаимосвязь.



ВОПРОСЫ





Параметр **streaming**



collect

Это метод, который является важнейшим компонентом работы с объектами LazyFrame, особенно при работе с большими наборами данных или потоковыми данными. Используется для выполнения запланированных операций над LazyFrame и сбора результатов в ожидающий вычислений DataFrame.



streaming

Это параметр в методе `collect`, который позволяет Polars обрабатывать и возвращать данные в потоковом режиме. При значении `True` метод `collect` обрабатывает данные по частям, а не загружает в память сразу весь набор данных.

Потоковый параметр

```
import polars as pl

lazy_df = pl.scan_csv("events.csv").filter(pl.col("price") > 100)

result_df = lazy_df.collect(streaming=True)
```

Потоковая обработка

Обработка чанков

Данные обрабатываются кусками.
В зависимости от операций и
порядка их применения это может
повлиять на конечные результаты,
особенно для операций,
требующих глобального
представления данных

Компромиссы с
производительностью

Потоковая обработка

Обработка чанков

Компромиссы с
производительностью

Необходимо учитывать влияние на
производительность и
сопоставлять его с
преимуществами эффективности
использования памяти.



ВОПРОСЫ





ВОПРОСЫ





Сравнение с pandas. Бенчмаркинг

Сравнение скорости выполнения

```
%%timeit

import pandas as pd

df = pd.read_csv("events.csv")

df_selected = df[['brand', 'event_type',
'price']]

df_filtered =
df_selected[df_selected['event_type'] == 'view']

result_df =
df_filtered.groupby('brand')['price'].mean().reset_index(name='average_price')

print(result_df)
```

Pandas: время выполнения: 3.82 s ± 59.7 ms per loop
(mean ± std. dev. of 7 runs, 1 loop each)

```
%%timeit

import polars as pl

result_df = (
    pl.scan_csv("events.csv")
    .select(['brand', 'event_type', 'price'])
    .filter(pl.col('event_type') == 'view')

    .group_by("brand").agg([pl.col("price").mean().alias("average_price")])
    .collect()

print(result_df)
```

Polars: время выполнения: 222 ms ± 15.2 ms per loop
(mean ± std. dev. of 7 runs, 1 loop each)



Работа с 
несколькими
файлами

Цикл в жадном режиме

```
df_list = []
for file in file_list:
    df_list.append(
        pl.read_csv(file)
    )

df = pl.concat(df_list)

output = (
    df
    .groupby('brand')
    .agg([pl.col("price").mean().alias("average_price")])
)
```

Цикл в ленивом режиме

```
df_list = []
for file in file_list:
    df_list.append(
        pl.scan_csv(file)
    )

df = pl.concat(df_list)

output = (
    df
    .groupby('brand')
    .agg([pl.col("price").mean().alias("average_price")])
).collect()
```

Упрощение

```
output = (  
    pl.scan_csv("./*.csv")  
    .groupby('brand')  
    .agg([pl.col("price").mean().alias("average_price")])  
).collect(streaming = True)
```



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



Задание



1. Что делает этот код?

```
df_list = []  
for file in file_list:  
    df_list.append(  
        pl.read_csv(file)  
    )  
  
df = pl.concat(df_list)
```

Задание

2. Исправьте код:

```
import polars as pl

lazy_df = pl.scan_csv("data.csv")

result = lazy_df.filter(pl.col("name") == "Alice").select(["id", "sales"])
result_df = result.collect()
print(result_df)
```



ВОПРОСЫ



Домашнее задание

Используйте набор данных "tips" и набор данных "flights" из Seaborn.

Инструкции:

1. Загрузите наборы данных "tips" и "flights" в отдельные Polars DataFrames.
2. Из набора данных 'flights' выберите столбцы 'year', 'month' и 'passengers'.
3. Создайте новый DataFrame из 'flights', который агрегирует общее количество пассажиров за год.
4. Объедините эти агрегированные данные с набором данных 'tips' по столбцу 'year' (примечание: для этого вам может понадобиться создать столбец 'year' в 'tips').
5. Конкатенируйте DataFrame 'tips' (отфильтрованный по времени 'Dinner') с первыми 50 строками 'flights' (подумайте, каким образом).
6. Отобразите первые 10 строк объединенного и конкатенированного DataFrame.

Заключение

